

CHAPTER 4

DESIGN AND METHODOLOGY

4.1 Research Design

This study employs a Developmental and Experimental Research Design to guide the design, fabrication, and cooperative-scale evaluation of CacaoScan. Developmental research is appropriate because the project produces an integrated AIoT artifact—a conveyor-fed sorting machine, cloud-hosted dual-model YOLOv8 pipeline, and mobile/web monitoring stack—rather than testing an existing commercial product. Experimental research complements this approach by enabling controlled measurement of detection accuracy, mechanical discharge reliability, and batch-cycle timing against predefined thresholds.

The experimental component focuses on comparing machine-assigned variety and quality grades with expert-labeled ground truth under PNS/BAFS 58:2019-aligned visual criteria. Controlled test runs using a 150-bean expert-labeled set will support confusion matrix analysis, mechanical sorting success rate computation, and User Acceptance Testing with five cooperative farmers. Field validation is planned at barangay-level post-harvest facilities in Barangay Cacao, Zamboanga City, consistent with the approved capstone capsule.

4.2 Developmental Methodology

Software and system integration follow an Agile Software Development Methodology organized into five two- to three-month sprints from July 2026 to June 2027. Each sprint delivers a testable increment that synchronizes mechanical feed tuning, dataset annotation, YOLOv8 training, cloud deployment, ESP32 firmware development, and dashboard integration until the conveyor, vision tower, inference service, and discharge modules operate as one pipeline.

Sprint 1 (July–September 2026) finalizes requirements, high-fidelity mobile and web UI prototypes, Supabase schema integration, and the target hybrid edge-cloud architecture. Sprint 2 (October–December 2026) gathers local Zamboanga cacao images, annotates datasets in Roboflow, and trains YOLOv8 Model A (variety) and Model B (quality) on a Ryzen 7 workstation with NVIDIA GPU acceleration. Sprint 3 (January–February 2027) transitions the

cardboard proof-of-concept into an acrylic/aluminum chassis and wires the ESP32-S3 with a six-servo actuation array and dual IR beam-break sensors.

Sprint 4 (March–April 2027) integrates the Industrial HD IP Camera, cloud AI API, and ESP32 discharge firmware, optimizing latency for cooperative throughput of 20 to 30 beans per minute across consecutive batch-of-three cycles. Sprint 5 (May–June 2027) conducts field testing and User Acceptance Testing with five cooperative farmers in Barangay Cacao, culminating in the final thesis defense.

[Figure 4.1 Agile Development Lifecycle of CacaoScan — diagram to be inserted manually]

4.3 System Architecture

CacaoScan follows a hybrid edge-cloud architecture composed of four interconnected layers, as illustrated in Figure 4.2. The physical machine layer contains the funnel hopper, vibratory feed assist, V-trough conveyor, IR staging gate, enclosed vision tower with three-pocket tray, Industrial HD IP Camera, and three-bin flipper-pusher discharge assembly. The IoT and edge control layer is governed by ESP32-S3 firmware written in C++ using the Arduino IDE, which receives cloud inference results through a WebSocket command channel and sequences six servos—gate, tilting distributor chute, three lateral pushers, and sorting flipper—with approximately 800 milliseconds of rail clearance between beans.

The application and cloud AI layer hosts the cacaoscan-ai-api FastAPI microservice for dual YOLOv8 inference on Hugging Face Inference Endpoints, a planned Node.js/Express synchronization layer, Cloudinary image archives, and Firebase Cloud Messaging alerts. The presentation layer consists of a React Native mobile application for field operators and a React JS responsive web dashboard for cooperative managers and system administrators. Supabase PostgreSQL stores batch logs, per-bean classifications, machine telemetry, and audit records with row-level security.

Table 4.1 summarizes the Capstone 1 implementation status. High-fidelity mobile and web user interfaces, the Supabase database schema, and mock inference/handshaking are built.

Physical six-servo integration, production ESP32 firmware, and full FastAPI deployment to Hugging Face Endpoints remain planned for Capstone 2.

[Figure 4.2 Hybrid Edge-Cloud System Architecture of CacaoScan — diagram to be inserted manually]

Table 4.1 Capstone 1 Implementation Status

Component	Status	Notes
Mobile app (React Native / Expo)	Built	Batch control, vision feed, auth, Supabase realtime
Web dashboard (React / Vite)	Built (UI shell)	Login and scaffolded dashboard modules only
Supabase schema (profiles, batches, classifications, machines)	Built	ERD and RLS policies defined
Mock ESP32 / inference handshaking	Built	WebSocket simulator for Capstone 1 testing
cacaoscan-ai-api (FastAPI + YOLOv8)	Planned	Separate repo; training started on Ryzen 7 + NVIDIA GPU
ESP32-S3 production firmware (C++)	Planned	PWM servos + digital IR sensors
Hugging Face endpoint deployment	Planned	ONNX/TensorRT export after training
Physical 6-servo machine integration	Planned	Acrylic/aluminum chassis fabrication
Manager analytics and PDF certificate export	Planned	Web dashboard modules under construction
MLOps model retraining portal	Planned	Administrator module scheduled for Capstone 2

4.4 Data Collection Methods

Data collection supports both AI model training and system evaluation. Image data include high-resolution tower captures of Criollo, Forastero, and Trinitario beans and visual defect classes—mold, shriveled, and slaty coloration—aligned with PNS/BAFS 58:2019 export-preparation intent. Sources combine Kaggle and Roboflow public datasets with at least 500 locally captured photos from cacao farms in Barangay Cacao. Target volume is 3,000 to 5,000 annotated images at 640 by 640 pixels with stratified train-validation-test partitioning and augmentation in Roboflow.

Evaluation data include expert-labeled beans for the 150-bean controlled test set, mechanical sorting success logs, batch-cycle timing records, and Likert-scale User Acceptance Testing responses from five cooperative farmers using the WMSU weighted mean interpretation scale. Certified cacao graders or DA/BFAR experts will validate ground-truth labels on a subset of annotated images before model evaluation.

4.5 Requirements Analysis

Requirements analysis documents the functional and data interactions of CacaoScan prior to full Capstone 2 integration. Figures 4.3 through 4.7 present the context diagram, data flow diagrams, system flowchart, and use case diagram. Narrative explanations below describe each figure so the diagrams can be drawn manually to match the approved system design.

4.5.1 Context Diagram

[Figure 4.3 Context Diagram of CacaoScan — diagram to be inserted manually]

The context diagram places CacaoScan at the center as a single automated sorting process. The Farmer (Operator) sends batch commands and receives live sorting feedback through the mobile application. The Manager (Cooperative Admin) receives analytics, certificate exports, and batch summaries through the web dashboard. The Industrial HD IP Camera supplies RTSP image streams to the cloud inference service. DA or BFAR expert graders provide

ground-truth validation for training and evaluation datasets. Supabase acts as the persistent cloud data store for batches, classifications, and audit logs.

4.5.2 Data Flow Diagram

[Figure 4.4 DFD Level 0 of CacaoScan — diagram to be inserted manually]

[Figure 4.5 DFD Level 1 of CacaoScan — diagram to be inserted manually]

DFD Level 0 shows major data flows between external entities and the CacaoScan sorting system, including batch commands, RTSP frames, inference results, and certification reports. DFD Level 1 decomposes the system into five processes: (1) stage batch of three using IR counting, (2) capture tower image, (3) execute cloud dual YOLOv8 inference, (4) discharge and sort via ESP32-S3 actuators, and (5) log results and notify clients through Supabase and Firebase Cloud Messaging. Data stores D1 (Batches) and D2 (Classifications) persist session state and per-bean records.

4.5.3 System Flowchart

[Figure 4.6 System Processing Flowchart of CacaoScan — diagram to be inserted manually]

The system flowchart begins when the operator loads dried fermented beans into the funnel hopper and starts a batch from the mobile application. Beans travel through the V-trough conveyor until the IR beam-break sensor confirms three beans at the staging gate. The vision tower captures one multi-pocket image and transmits it to the cloud AI API. Model A assigns variety labels and Model B assigns quality grades for each pocket position using PNS/BAFS 58:2019-aligned criteria. Decision fusion prioritizes Model B output for bin routing while retaining Model A labels for analytics. The ESP32-S3 executes flipper-and-pusher discharge, logs each bean to Supabase, and repeats until the batch completes or the operator pauses the run.

4.5.4 Use Case Diagram

[Figure 4.7 Use Case Diagram of CacaoScan — diagram to be inserted manually]

UC-1 Start, Pause, or Resume Sorting Batch

Actor: Farmer (Operator)

Implementation Status: Built (Capstone 1)

Description: The system shall allow the operator to create a batch session, start machine sorting, pause processing, and resume from the mobile Home screen.

Fit Criterion: A batch record is created in Supabase with status active or paused, live counters update over realtime channels, and the ESP32 mock receives START or PAUSE WebSocket commands.

Use Case Script:

1. Open the CacaoScan mobile app and sign in with a verified farmer account.
2. On the Home tab, enter batch name, harvest date, and optional target bean count.
3. Tap Start Batch to insert a row in the batches table and send {command: "START"} to the ESP32 WebSocket endpoint.
4. Tap Pause to set batch status to paused, save duration_seconds, and send {command: "PAUSE"}.
5. Tap Resume to set status back to active and send {command: "START"} again.
6. Swipe to Stop to complete the batch (or delete an empty session) and send {command: "STOP"}.

UC-2 View Live Vision Classification

Actor: Farmer (Operator)

Implementation Status: Built (Capstone 1)

Description: The system shall display live variety and quality results for each detected bean on the Vision tab.

Fit Criterion: The Vision screen shows the latest BEAN_DETECTED event with variety, quality, confidence bars, and a rolling history list while the machine state is RUNNING.

Use Case Script:

1. Ensure an active batch is running from the Home tab.
2. Navigate to the Vision tab.
3. Observe the LIVE indicator and machine state (RUNNING, PAUSED, or STOPPED).
4. When the ESP32 mock or hardware emits a BEAN_DETECTED message, the card updates with variety, quality, and confidence values.
5. Each result is inserted into the classifications table when a batch_id is active.

UC-3 Flag AI Classification Error

Actor: Farmer (Operator)

Implementation Status: Built (Capstone 1)

Description: The system shall allow the operator to flag an incorrect AI result for MLOps review.

Fit Criterion: The selected classification record sets is_flagged to true and stores farmer_correction text; the Report Error control becomes disabled after submission.

Use Case Script:

1. Open History and select a completed or active batch.
2. Scroll to the bean record that appears misclassified.
3. Tap Report Error on the classification row.
4. Confirm the report; Supabase updates is_flagged and farmer_correction for that record.

5. The UI displays a Reported state to prevent duplicate flags.

UC-4 View Analytics Dashboard

Actor: Manager (Cooperative Admin)

Implementation Status: Planned (Capstone 2)

Description: The system shall provide batch statistics and historical trends on the web dashboard.

Fit Criterion: The manager views aggregated variety and quality counters per batch and date range once the Analytics Board module is implemented.

UC-5 Export Quality Certificate PDF

Actor: Manager (Cooperative Admin)

Implementation Status: Planned (Capstone 2)

Description: The system shall generate a downloadable PDF quality certificate for a completed batch.

Fit Criterion: A PDF summary is exported containing batch metadata, grade distribution, and harvest date after the certificate module is deployed.

UC-6 Retrain or Deploy AI Models

Actor: System Administrator (Developer)

Implementation Status: Planned (Capstone 2)

Description: The system shall allow administrators to manage YOLOv8 model versions and monitor inference health.

Fit Criterion: Updated model weights are exported through ONNX/TensorRT and deployed to Hugging Face Inference Endpoints via the MLOps portal.

UC-7 Monitor Hardware Telemetry

Actor: System Administrator (Developer)

Implementation Status: Planned (Capstone 2)

Description: The system shall report machine online status and last heartbeat from the ESP32-S3.

Fit Criterion: The machines table reflects `is_online` and `last_heartbeat` once production firmware telemetry is integrated.

4.6 Requirement Specification

The requirement specification defines functional and non-functional criteria for CacaoScan. Capstone 1 implements farmer-facing mobile capabilities and database logging; manager analytics, PDF export, cloud AI deployment, and production firmware are scheduled per Sprints 4–5. Functional requirements below are labeled Built or Planned to match Table 4.1.

4.6.1 Functional Requirements

Authentication and Profile Management — [Built] Secure registration, login, email OTP verification, password reset, and role-based profiles (farmer or admin) linked to farm location and optional machine ID.

Batch Session Management — [Built] Create, pause, resume, and complete sorting batches with harvest date, target bean count, live variety/quality counters, and duration tracking via Supabase realtime.

Live Vision and Classification Feed — [Built] Display Model A variety and Model B quality results with confidence bars and recent classification history on the Vision tab.

Manual Override — [Built] Dedicated manual override screen for operator intervention when mechanical or inference anomalies occur.

Flag Incorrect AI Results — [Built] Mark individual classification rows with `is_flagged` and `farmer_correction` from batch detail view.

Manager Analytics Dashboard — [Planned] Web modules for batch ledger review, yield analytics, and trend charts.

PDF Quality Certificate Export — [Planned] Downloadable batch certificate for cooperative buyers and internal records.

MLOps and Model Deployment — [Planned] Administrator tools for flagged-review queue, model version management, and Hugging Face endpoint updates.

Hardware Telemetry Monitoring — [Planned] Machine online status and heartbeat reporting to the machines table.

4.6.2 User Characteristics

Table 4.2 User Roles and Access Levels

User Role	Priority	Platform	Features
Farmer (Operator)	Primary	Mobile app	Start/pause batch, live vision, flag errors, manual override
Manager (Cooperative Admin)	Secondary	Web dashboard	Analytics, batch ledger, PDF certificates (planned)
System Administrator (Developer)	Key	Web MLOps + admin	Model retraining, hardware telemetry, user management (planned)

In the implemented Supabase schema, profiles.role accepts farmer or admin. The Farmer actor maps to farmer. The Manager and System Administrator actors map to admin with differentiated dashboard module access once Capstone 2 web features are completed.

4.6.3 Non-Functional Requirements

Technical Requirements — The system uses React Native (Expo) for mobile, React JS (Vite) for web, FastAPI for AI inference (planned), Node.js/Express for backend synchronization (planned), Supabase PostgreSQL for persistence, Cloudinary for image archives (planned), and ESP32-S3 firmware for actuation (planned). Cooperative facilities require stable Wi-Fi for cloud inference without an on-site inference laptop.

Performance Requirements — Target detection accuracy above 85 percent for Models A and B, mechanical sorting success rate of 95 percent during the 150-bean test set, batch-of-three vision-and-discharge cycles with approximately 800 milliseconds rail clearance, cooperative throughput of 20 to 30 beans per minute, and mobile UI updates over Supabase realtime channels.

Security Requirements — Supabase Auth with bcrypt password hashing, email OTP multi-factor authentication, login audit logging, row-level security on all tables, failed-login lockout after five attempts, generic authentication error messages, and HTTPS/TLS for all cloud API traffic as documented in the mobile security schema.

Assumptions and Dependencies — Capstone 1 uses mock ESP32 WebSocket handshaking when physical firmware is unavailable; full hardware integration depends on Sprint 3 fabrication; cloud AI endpoints depend on completed YOLOv8 training in Sprint 2; field UAT depends on cooperative access in Barangay Cacao.

4.7 System Design

Database design follows a relational model implemented in Supabase PostgreSQL with row-level security. Figure 4.8 presents the entity-relationship diagram. Core entities are profiles, batches, classifications, machines, and login_audit_logs, matching the schema deployed in the cacaoScan-mobile repository.

[Figure 4.8 Entity-Relationship Diagram of CacaoScan — diagram to be inserted manually]

4.7.1 Database Field Tables

Table 4.3 Database Fields — profiles

Field	Type	Key	Nullable	Description
id	UUID	PK	No	References auth.users(id)
first_name	TEXT		No	User given name
last_name	TEXT		No	User surname
farm_location	TEXT		No	Farm or barangay location
role	TEXT		No	farmer or admin
contact_number	TEXT		No	Contact number (default empty)
linked_machine_id	TEXT	FK	Yes	References machines(machine_id)
created_at	TIMESTAMPTZ		Yes	Record creation timestamp
updated_at	TIMESTAMPTZ		Yes	Last profile update

Table 4.4 Database Fields — batches

Field	Type	Key	Nullable	Description
id	UUID	PK	No	Batch identifier
user_id	UUID	FK	No	Owner auth.users(id)
batch_name	TEXT		No	Session label
harvest_date	DATE		No	Harvest date of beans
target_bean_count	INTEGER		No	Optional target (0 = continuous)
status	TEXT		No	active, paused, or completed

criollo_count	INTEGER		No	Model A variety counter
forastero_count	INTEGER		No	Model A variety counter
trinitario_count	INTEGER		No	Model A variety counter
export_grade_count	INTEGER		No	Model B quality counter
needs_drying_count	INTEGER		No	Model B quality counter
rejected_count	INTEGER		No	Model B quality counter
total_beans	INTEGER		No	Aggregate throughput
duration_seconds	INTEGER		No	Elapsed session time
started_at	TIMESTAMPTZ		Yes	Batch start time
completed_at	TIMESTAMPTZ		Yes	Batch completion time
created_at	TIMESTAMPTZ		Yes	Record creation timestamp

Table 4.5 Database Fields — classifications

Field	Type	Key	Nullable	Description
id	UUID	PK	No	Record identifier
batch_id	UUID	FK	No	References batches(id)

variety	TEXT		No	criollo, forastero, or trinitario
variety_confidence	REAL		No	Model A confidence score
quality	TEXT		No	export_grade, needs_drying, or rejected
quality_confidence	REAL		No	Model B confidence score
classified_at	TIMESTAMPTZ		Yes	Timestamp of inference
image_url	TEXT		Yes	Cloudinary tower image URL (planned)
is_flagged	BOOLEAN		Yes	Farmer-reported error flag
farmer_correction	TEXT		Yes	Optional correction note

Table 4.6 Database Fields — machines

Field	Type	Key	Nullable	Description
machine_id	TEXT	PK	No	Hardware identifier
master_pin	TEXT		No	Pairing/authentication pin
owner_id	UUID	FK	Yes	References profiles(id)
is_online	BOOLEAN		Yes	Telemetry online flag

last_heartbeat	TIMESTAMPTZ		Yes	Last ESP32 heartbeat
----------------	-------------	--	-----	----------------------

Table 4.7 Database Fields — login_audit_logs

Field	Type	Key	Nullable	Description
id	UUID	PK	No	Audit record identifier
user_email	TEXT		No	Account email audited
login_status	TEXT		No	success, failed, locked, otp_sent, etc.
ip_address	TEXT		Yes	Client address metadata
user_agent	TEXT		Yes	Browser or app agent string
device_info	TEXT		Yes	Device descriptor
session_secure_flag	BOOLEAN		Yes	HTTPS-only session indicator
session_httponly_flag	BOOLEAN		Yes	HttpOnly session indicator
session_samesite	TEXT		Yes	SameSite cookie policy
is_suspicious	BOOLEAN		Yes	Suspicious login flag
suspicious_reason	TEXT		Yes	Reason for suspicion

created_at	TIMESTAMPTZ		Yes	Event timestamp
------------	-------------	--	-----	--------------------

4.7.2 Hardware Interface

Hardware interface design defines how the ESP32-S3 controller coordinates mechanical staging and discharge with the cloud inference pipeline. Capstone 1 validates the command protocol using a Node.js WebSocket mock simulator; production C++ firmware in the Arduino IDE is planned for Capstone 2.

Actuation uses six PWM-controlled servos: (1) staging gate, (2) tilting distributor chute, (3–5) three SG90 lateral pushers for pocket discharge, and (6) MG996R three-way sorting flipper for Export-Grade, Needs Drying, and Rejected bins. Dual IR beam-break sensors on digital input pins count beans at the staging gate and confirm batch-of-three release before the vision tower capture signal is sent.

Communication between the mobile application and the controller uses JSON WebSocket messages. Operator commands are {command: "START"}, {command: "PAUSE"}, and {command: "STOP"}. The controller responds with {type: "STATUS", state: "RUNNING"|"PAUSED"|"STOPPED"} and {type: "BEAN_DETECTED", data: {variety, variety_confidence, quality, quality_confidence}} after each inference cycle. In the target Capstone 2 design, the AI server will trigger discharge through the same low-latency WebSocket channel once cloud YOLOv8 results are available.

4.8 Tools and Technology

Development tools include Visual Studio Code, GitHub, Arduino IDE, Roboflow, Ultralytics YOLOv8, Expo CLI, and Vite. Model training uses a Ryzen 7 workstation with NVIDIA GPU acceleration. Deployment targets include Hugging Face Inference Endpoints, Supabase cloud, and cooperative Wi-Fi standalone operation at the client site.

4.9 Evaluation and Testing

Evaluation and testing proceed in three tracks aligned with Sprint 5. AI performance testing uses separate confusion matrices and per-class precision, recall, and F1-score for Models

A and B on the 150-bean expert-labeled test set. Mechanical reliability testing measures sorting success rate and batch-cycle completion during controlled runs. User Acceptance Testing engages five cacao farmers from local cooperatives in Barangay Cacao using a Likert scale targeting a weighted mean above 4.20.

Third-party consultants—certified cacao graders or DA/BFAR experts—validate ground-truth labels and compare machine-assigned grades with manual expert sorting. Results above stated thresholds will be interpreted as evidence that CacaoScan is a feasible cooperative-scale AIoT solution for Zamboanga post-harvest sorting. Actual numeric results will be reported after Capstone 2 field testing; Capstone 1 documents the procedures and acceptance criteria.

REFERENCES

- [1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>. [Accessed: Jun. 19, 2026].
- [2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637-646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.
- [3] Philippine Bureau of Agriculture and Fisheries Standards, Philippine National Standard for Cacao Beans, PNS/BAFS 58:2019. Quezon City, Philippines: Department of Agriculture, 2019.
- [4] Supabase Inc., "Supabase Documentation — Row Level Security." [Online]. Available: <https://supabase.com/docs/guides/auth/row-level-security>. [Accessed: Jun. 19, 2026].